16 Advanced concurrent programming

This chapter covers

- The Fork/Join API
- Work-stealing algorithms
- Concurrency and functional programming
- Under the hood with Kotlin coroutines
- Clojure concurrency
- Software transactional memory
- Agents

In this chapter, we will bring together several themes from earlier chapters. In particular, we will weave together the functional programming concepts from previous chapters with the Java concurrency libraries from chapter 6. Our non-Java languages get included as well, with some novel concurrency aspects of both Kotlin and Clojure appearing later in the chapter.

NOTE The concepts in this chapter, such as coroutines and agents (aka actors), are also increasingly part of the landscape of Java concurrency.

We'll kick off with a slight oddity: the Java Fork/Join API. This framework allows a certain class of concurrent problems to be handled more efficiently than the executors we saw in chapter 6.

16.1 The Fork/Join framework

As we discussed in chapter 7, processor speeds (or, more properly, transistor counts on CPUs) have increased hugely in recent years. I/O performance has not had the same remarkable improvement, and so the end result is that waiting for I/O is now a common situation. This suggests that we could make better use of the processing capabilities inside our computers. The Fork/Join (F/J) framework is an attempt to do just that.

F/J is all about automatic scheduling of tasks on a thread pool that is invisible to the user. To do this, the tasks must be able to be broken up in a way that the user specifies.

In many applications, F/J has a notion of "small" and "large" tasks that is very natural for the framework.

Let's take a quick look at some of the headline facts and fundamentals related to F/J:

- The framework introduces a new kind of executor service, called a `ForkJoinPool`.
- `ForkJoinPool` handles a unit of concurrency (the `ForkJoinTask`) that is "smaller" than a `Thread`.
- `ForkJoinTask` can be scheduled in a more lightweight manner by the `ForkJoinPool`.
- F/J makes use of the following two kinds of tasks (both represented as instances of `ForkJoinTask`):
  - "Small" tasks are those that can be performed straightaway without consuming too much processor time.
  - "Large" tasks are those that need to be split up (possibly more than once) before they can be directly performed.
- The framework provides basic methods to support the splitting up of large tasks.
- The framework has automatic scheduling and rescheduling.

One key feature of the framework is that it's expected that these lightweight tasks may well spawn other instances of `ForkJoinTask`, which will be scheduled on the same thread pool that executed their parent task. This pattern is sometimes called *divide and conquer*.

We'll start with a simple example of using the F/J framework, then briefly touch on the features of problems that are well suited to this type of parallel processing approach. We will then discuss the feature called "work-stealing" as used in F/J and its relevance in a wider context. The best way to get started with F/J is with an example.

6.1.1 A simple F/J example

As a simple example of what the F/J framework can do, consider the following case: we have a number of transaction objects created at various times. The class we'll use to represent them is `Transaction`, as shown here, which is an evolution of the `TransferTask` class we met in chapters 5 and 6:

```
public class Transaction implements Comparable<Transaction> {
    private final Account sender;
    private final Account receiver;
    private final int amount;
    private final long id;
    private final LocalDateTime time;
```

```
    private static final AtomicLong counter = new AtomicLong(1);

    Transaction(Account sender, Account receiver,
            int amount, LocalDateTime time) {
      this.sender = sender;
      this.receiver = receiver;
      this.amount = amount;
      this.id = counter.getAndIncrement();
      this.time = time;
    }

    public static Transaction of(Account sender, Account receiver,
                         int amount) {
      return new Transaction(sender, receiver,
                        amount, LocalDateTime.now());
    }

    @Override
    public int compareTo(Transaction other) {
      return Comparator.nullsFirst(LocalDateTime::compareTo)
                  .compare(this.time, other.time);
    }

    // Getter and other methods (equals, hashcode, etc) elided
  }
```

We want to obtain a list of transactions, sorted by time. To achieve this, we'll use F/J as a multithreaded sort—in fact a variant of the *MergeSort* algorithm.

Our example uses `RecursiveAction`, which is a specialized subclass of `ForkJoinTask`. It is simpler than general `ForkJoinTask` because it's explicit about not having any overall result (the transactions will be reordered in place), and it emphasizes the recursive nature of the tasks.

The `TransactionSorter` class provides a way of ordering a list of updates using the `compareTo()` method on `Transaction` objects. The `compute()` method (which you have to implement because it's abstract in the `RecursiveAction` superclass) basically orders an array of transactions by their creation time, as shown in the next listing.

Listing 16.1 Sorting with a `RecursiveAction`

```
public class TransactionSorter extends RecursiveAction {
    private static final int SMALL_ENOUGH = 32;                    (1)
    private final Transaction[] transactions;
    private final int start, end;
    private final Transaction[] result;

    public TransactionSorter(List<Transaction> transactions) {
        this(transactions.toArray(new Transaction[0]),
             0, transactions.size());
    }

    public TransactionSorter(Transaction[] transactions) {
        this(transactions, 0, transactions.length);
    }

    public TransactionSorter(Transaction[] txns, int start, int end) {
        this.start = start;
        this.end = end;
        this.transactions = txns;
        this.result = new Transaction[this.transactions.length];
    }

    /**
     * This method implements a simple Mergesort. Please consult a suitable
     * textbook if you are interested in the implementation details.
     *
     * @param left
     * @param right
     */
    private void merge(TransactionSorter left, TransactionSorter right) {
        int i = 0;
        int lCount = 0;
        int rCount = 0;

        while (lCount < left.size() && rCount < right.size()) {
            int comp = left.result[lCount].compareTo(right.result[rCount]);
            result[i++] = (comp < 0)
                    ? left.result[lCount++]
                    : right.result[rCount++];
        }

        while (lCount < left.size()) {
            result[i++] = left.result[lCount++];
        }

        while (rCount < right.size()) {
```

```
            result[i++] = right.result[rCount++];
        }
    }

    public int size() {
        return end - start;
    }

    public Transaction[] getResult() {
        return result;
    }

    @Override
    protected void compute() {                                    ❷
        if (size() < SMALL_ENOUGH) {
            System.arraycopy(transactions, start, result, 0, size());
            Arrays.sort(result, 0, size());
        } else {
            int mid = size() / 2;
            TransactionSorter left =
                new TransactionSorter(transactions, start, start + mid);
            TransactionSorter right =
                new TransactionSorter(transactions, start + mid, end);
            invokeAll(left, right);

            merge(left, right);
        }
    }
}
```

❶ 32 or fewer sorted serially

❷ The method defined in RecursiveAction

To use the sorter, you can drive it with some code like that shown next, which will generate some transactions and shuffle them, before passing them to the sorter. The output is the reordered updates:

```
var transactions = new ArrayList<Transaction>();
var accs = new Account[] {
            new Account(1000),
            new Account(1000)};

for (var i = 0; i < 256; i = i + 1) {
```

```
  transactions.add(Transaction.of(accs[i % 2], accs[(i + 1) % 2], 1));
  Thread.sleep(1);
}
Collections.shuffle(transactions);

var sorter = new TransactionSorter(transactions);
var pool = new ForkJoinPool(4);

pool.invoke(sorter);

for (var txn : sorter.getResult()) {
  System.out.println(txn);
}
```

The promise of F/J seems tantalizing, but in practice, not every problem is easily reduced to as simple a form as the multithreaded `MergeSort` we've just discussed.

This is an example of the antipattern *Easy Cases Are Easy*, in which developers can be seduced by a simple-seeming technology that allows an easy task to be achieved with very little effort but disguises the fact that the technology does not scale or generalize well to the less-easy cases. We should say something about the sorts of problems that are likely to be tractable by the use of F/J, and those problems for which another approach will likely be better.

6.1.2 Parallelizing problems for F/J

Here are some examples of problems well suited to the F/J approach:

- Simulating the motion of large numbers of simple objects (e.g., particle effects)
- Log file analysis
- Data operations where a quantity is calculated from aggregated inputs (e.g., map-reduce operations)

Another way of looking at this is to say that a good problem for F/J is one that can be broken up, as shown in figure 16.1.

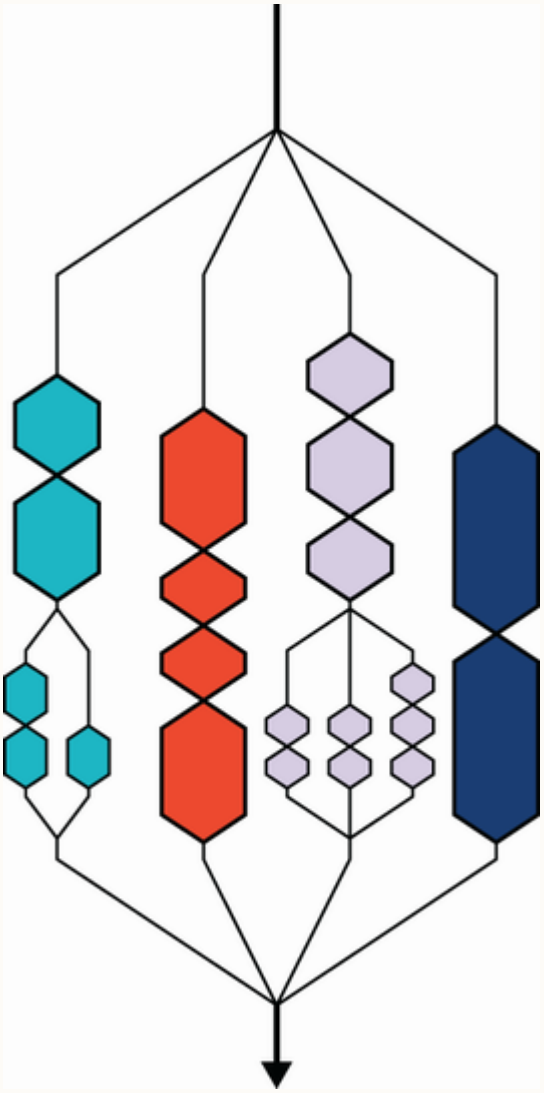


Figure 16.1 Fork and join

One practical way of determining whether a problem is likely to reduce well is to apply the following checklist to the problem and its subtasks:

- Can the problem's subtasks work without explicit cooperation or synchronization between the subtasks?
- Do the subtasks calculate some value from their data without altering it (i.e., are they pure functions)?
- Is divide and conquer natural for the subtasks?

If the answer to the preceding questions is "Yes!" or "Mostly, but with edge cases," your problem may well be amenable to a F/J approach. If, on the other hand, the answer to those questions is "Maybe" or "Not really," you may well find that F/J performs poorly, and a different approach may work better.

Designing good multithreaded algorithms is hard, and F/J doesn't work in every circumstance. It's very useful within its own domain of applicability, but in the end, you have to decide whether your problem fits within the framework. If not, you must be prepared to

develop your own solution, which probably means building on top of the superb toolbox of `java.util.concurrent`.

6.1.3 Work-stealing algorithms

`ForkJoinTask` is the superclass of `RecursiveAction`. It's a generic class in the return type of an action (so `RecursiveAction` extends `ForkJoinTask<Void>`). This makes `ForkJoinTask` very suitable for map-reduce approaches that boil down a dataset and either return a result or act via side effect (as is the case for `RecursiveAction`).

Objects of type `ForkJoinTask` are scheduled on a `ForkJoinPool`, which is a new type of executor service designed specifically for these lightweight tasks. The service maintains a list of tasks for each thread, and if one task finishes, the service can reassign tasks from a fully loaded thread to an idle one. We can see this happening in figure 16.2

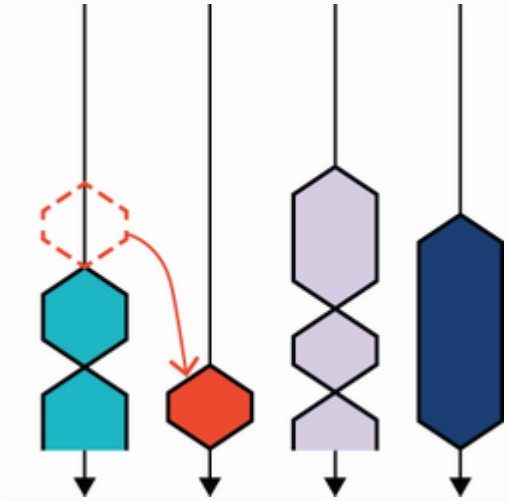


Figure 16.2 Work-stealing: When the second thread completes its task, the service reassigns a task from the still-busy first thread to the second.

Without this *work-stealing algorithm*, scheduling problems could arise related to the two sizes of tasks. In general, the two sizes of tasks could take very different lengths of time to run.

For example, one thread may have a run queue consisting of only small tasks, whereas another may have only large tasks. If the small tasks run five times faster than large tasks, the thread with only small tasks may well find itself idle before the large-task thread finishes.

WARNING Work-stealing depends on the assumption that the tasks are independent of each other. If this assumption is not valid, the results of calculations could be different from run to run.

Work-stealing has been implemented precisely to work around this problem and allow all the pool threads to be used throughout the lifecycle of the F/J job. It's completely automatic, and you don't need to do anything specific to reap the benefits of work-stealing. It's another example of the runtime environment doing more to help developers manage concurrency, rather than making it a manual task. The documentation also makes it explicit: `ForkJoinPool` may also be appropriate for use with event-style tasks that are never joined.

NOTE `ForkJoinPool` is also used in popular Java/JVM libraries, such as the Akka system of actor-based concurrency in Scala and Java.

To interact with a `ForkJoinPool`, the class exposes the following primary methods:

- `execute()` —Starts an asynchronous execution
- `invoke()` —Starts execution and awaits the result
- `submit()` —Starts execution and returns a future for the result

Since Java 8, the runtime has included a common pool, which is accessed via `ForkJoinPool.commonPool()`. This is provided primarily for its work-stealing capabilities —there is not a lot of expectation that many programs will use it for recursive decomposition.

The common pool has a number of configurable properties that can be set to control things like the parallelism level (i.e.. how many threads to use) and the thread factory class used to create new threads for the common pool.

16.2 Concurrency and functional programming

In chapter 5, we met the concept of immutable objects and showed that they are extremely useful for concurrent programming because they sidestep the problem of *shared mutable state*, which is at the heart of so many concurrency problems. So, we might guess that functional techniques that utilize immutability are an important tool for building concurrent applications. This is true, but a small extension to immutability is also relevant to concurrent programming.

16.2.1 CompletableFuture revisited

In chapter 6, we met the `CompletableFuture` class. This type is not immutable, but it has a very simple state model, described next:

- It starts in an uncompleted state.

- Any attempt to `get()` a value from it will block.
- At some later time, a publication event occurs.
- This sets the value and passes it to any threads blocking on `get()`.
- The published value is now immutable.

Figure 16.3 shows the future and the publication event.

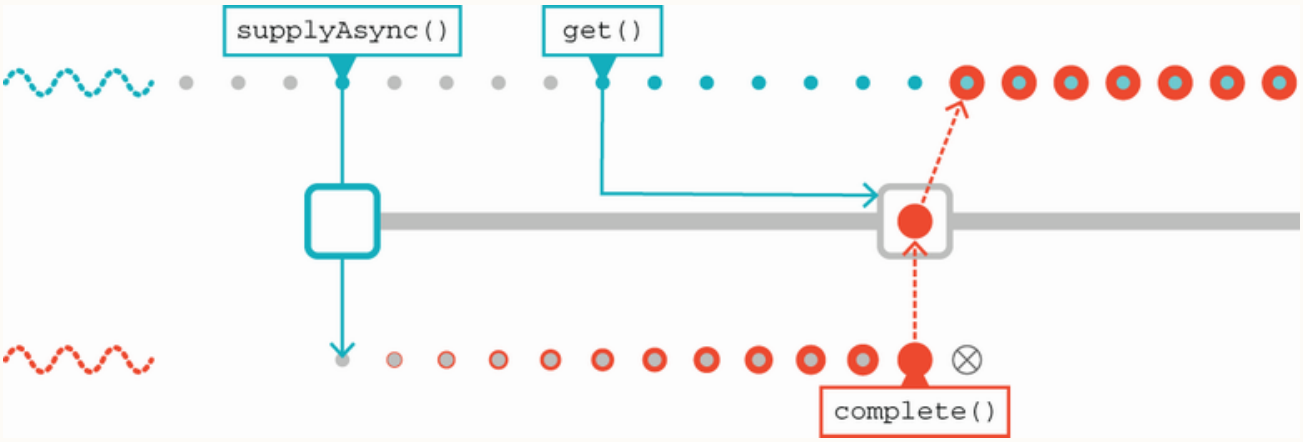


Figure 16.3 Event publication sets the value and passes it to any threads blocking on `get()`.

One big strength of `CompletableFuture` is that it is possible to compose functions with the result, and the result will be lazily evaluated, that is, the function will not be executed until the value has arrived.

This function composition can occur either synchronously or asynchronously. This is perhaps easiest to see by running a few examples. Let's reuse the idea of the `NumberService` from chapter 6 and use a dummy implementation for it, for example:

```
public class NumberService {
   public static long findPrime(int n) {
      try {
         Thread.sleep(5_000);
      } catch (InterruptedException e) {
         throw new CancellationException("interrupted");
      }
      return 42L;
   }
}
```

This obviously doesn't actually compute primes, but it is good enough to demonstrate the threading behavior, which is our goal. We need a bit of code to drive this, as shown next:

```
var n = 1000;
var future =
  CompletableFuture.supplyAsync(() -> {                    ❶
  System.out.println("Starting on: "+ Thread.currentThread().getName());
  return NumberService.findPrime(n);
});
var f2 = future.thenApply(l -> {                           ❷
  System.out.println("Applying on: "+ Thread.currentThread().getName());
  return l * 2;
});
var f3 = future.thenApplyAsync(l -> {                      ❸
  System.out.println("Async on: "+ Thread.currentThread().getName());
  return l * 3;
});

try {
  System.out.println("F2: "+ f2.get());
  System.out.println("F3: "+ f3.get());
} catch (InterruptedException | ExecutionException e) {
  e.printStackTrace();
}
```

❶ Supplies the computation to run asynchronously

❷ Supplies the function to apply to the result of the asynchronous computation

❸ Supplies another function to apply asynchronously to the result

When we run this code, we get some output like this:

```
Starting up on thread: ForkJoinPool.commonPool-worker-19
Applying on thread: ForkJoinPool.commonPool-worker-19
Applying async on thread: ForkJoinPool.commonPool-worker-5
F2: 84
F3: 126
```

The future `f2`, which uses `thenApply()`, executes on the same thread as `future`, whereas `f3` (which uses `thenApplyAsync()`) executes on a different thread in the pool.

You might notice that, by default, execution of `CompletableFuture` code all uses the common pool. This pool shows up with the name of `ForkJoinPool.commonPool`, as can be seen in the previous output.

Under some circumstances, developers may want to use an alternate thread pool. For example, the common pool isn't configurable in terms of the maximum number of threads that can be used, and this may not be acceptable for certain workloads. Fortunately, the factory methods of `CompletableFuture`, such as `supplyAsync()`, come with overloads that take an explicit `Executor` argument. This allows the future to run on a specific threadpool instead.

As well as the `thenApply()` method, `CompletableFuture` also provides `thenCompose()`. Some developers find the difference between the two confusing, so let's take a moment to explain.

Recall that `thenApply()` takes as its argument a `Function` that maps `T -> U`. This function is applied synchronously on whichever thread the `CompletableFuture` is running on after the original future completes.

On the other hand, `thenCompose()` takes a `Function` that maps `T -> CompletableFuture<U>` (the actual return type is `CompletionStage<U>` rather than `CompletableFuture<U>`, but let's gloss over that detail for now). This is, effectively, an asynchronous function (and it could run on a different thread). Let's look at a concrete example:

```
Function<Long, CompletableFuture<Long>> f = l ->
  CompletableFuture.supplyAsync(() -> {
    System.out.println("Applying on thread: " +
                       Thread.currentThread().getName());
    return l * 2;
  });
```

We *could* pass this function to `thenApply()`, but the result would be a `CompletableFuture<CompletableFuture<Long>>`. Instead, `thenCompose()` flattens the result back to a `CompletableFuture<Long>`. This is similar to how the `flatMap()` method works in the Java Streams API—it applies a function that returns `Stream<T>` to a stream object, but instead of returning a `Stream<Stream<T>>`, the separate streams are flattened and joined together into a single stream.

`CompletableFuture` also supports `join()`, which essentially works like a thread join but returns a value. It is also possible to "join together" futures by having code that is intended to run after either (or both) of them have finished. For example:

```
var n = 1000;
var future = CompletableFuture.supplyAsync(() -> {
   System.out.println("Starting up: "+ Thread.currentThread().getName());
   return NumberService.findPrime(n);
});

var future2 = CompletableFuture.supplyAsync(() -> {
   System.out.println("Starting up: "+ Thread.currentThread().getName());
   return NumberService.findPrime(n);
});

Runnable dontKnow = () -> System.out.println("One of the futures finished");
future.runAfterEither(future2, dontKnow);
```

We now want to bring together the discussion of functional programming in the previous chapter with the ideas here.

If the functions that we want to apply to the result of a `CompletableFuture` are pure and do not depend upon anything but the input value, the action of applying a function to the future is the same as applying the function to the result. Put another way: if the future is thought of as a container type, which holds a value, then the container is "transparent" to the function that is applied to the value once it has arrived.

In particular, two of the major benefits of referential transparency (e.g., using pure functions) follow:

- Memoization
- Transportability

The first means that any pure function call can be replaced by an already-calculated value—we don't have to rerun the function call with the same arguments, because we already know what the answer is. Second, of course, if we're calculating a pure function, then it doesn't matter what thread it takes place on, so whether we've supplied a function to be applied synchronously or asynchronously won't affect the result.

NOTE Java, as already discussed, is quite an impure language, so many of these benefits will apply only if the programmer is careful to use pure functions and immutable data throughout.

While we're discussing concurrent functional programming, it seems pertinent to say something about *parallel streams*, which are an area of the Streams API that many devel-

opers misunderstand.

6.2.2 Parallel streams

In chapter 5, we met Amdahl's law, which is one of the fundamental results about concurrency as it pertains to *data parallelism*. This is the approach to concurrency that we often want to use when we have a large amount of bulk data that is pretty similar and all has to be processed in more or less the same way. In general, the data parallelism approach is useful if all of the following are true:

- You have a lot of data to process in the same (or a very similar) way.
- Ordering doesn't matter.
- Items are independent of each other.
- You can show a particular processing step is the bottleneck.

Parallel streams are a type of data parallelism that a lot of Java developers got quite excited about when they were included in Java 8. However, as we'll see, the reality has turned out rather differently to the initial hope. The API, shown here, seems simple enough:

```
// Just replace stream() with parallelStream()
List<String> origins = musicians
    .parallelStream()
    .filter(artist -> artist.getName().startsWith("The"))
    .map(artist -> artist.getNationality())
    .collect(toList());
```

Under the hood, the work is distributed using F/J framework and uses the work-stealing algorithm to spread out the computation over multiple cores. The following seems too good to be true:

- The work is managed by the framework.
- API aims to be explicit but unobtrusive.
- Distributed by data.
- `parallelStream()` allows the programmer to flip between sequential and parallel.
- "Free speedup."

In fact, it *is* too good to be true. The first and most obvious problem is Amdahl's law. To split a sequential task into a set of chunks that can be executed in parallel requires work

—that is, computation time. The more preparation and communication overhead, the less benefit multiple processors provide—this is the essence of Amdahl's law.

No general way exists to easily and reliably estimate the relative costs of splitting versus the linear operation cost. The framework passes the cognitive cost of deciding whether parallelism is actually worthwhile back up to the developer. This already sounds like "automatic parallelism" is not the nirvana that was promised. Instead, the end user must be acutely aware of many details that would ideally be abstracted away.

To take just one example: the splitting and recombining work must take place on a thread pool within the JVM. The more threads that the JVM creates, the more they compete for CPU time. The Streams API does not, a priori, know how many other instances of parallel streams exist within the current process. This leads to the following two possible, equally unpalatable strategies:

- Create a new, dedicated thread pool for each invocation of parallel streams.
- Create a single instance of a thread pool (private to the JVM), and make all invocations of parallel streams use it.

The first option leads to a potentially unbounded creation of threads, which will ultimately starve or crash a JVM. For this reason, in Java 8, there is a single shared thread pool underlying parallel streams: `ForkJoinPool.commonPool()`. This choice leads to potential contention on a shared resource (which, as we have seen in chapter 7, is the real root of a great many performance problems).

There is a workaround: if you execute a parallel stream as a task on a `ForkJoinPool`, it will execute there and not use the common pool, like this:

```
// Use a custom pool
var forkJoinPool = new ForkJoinPool(4);
List<String> origins2 = forkJoinPool.submit(() -> musicians
    .parallelStream()
    .filter(artist -> artist.getName().startsWith("The"))
    .map(artist -> artist.getNationality())
    .collect(toList())).get();
forkJoinPool.shutdown();
```

Note that the `forkJoinPool` must be explicitly shut down, or it will persist in memory waiting for new tasks and thus leak memory and threads.

In general, the best advice that can be given about parallel streams is not to apply the parallelism blindly. Instead, actually show you have a use case for it. As always, this is done by measuring and by showing that the stream operations are indeed the bottleneck first, before attempting to see whether parallel streams will help.

Unfortunately, there is no general class of problem where parallel streams might be expected to help. Each case must be examined and tested from first principles. Only then can you try to apply parallelism to the stream and prove with data that a worthwhile improvement can be achieved.

16.3 Under the hood with Kotlin coroutines

As we introduced in chapter 9, Kotlin provides an alternative to the `Thread` model for concurrency with *coroutines*. Coroutines can be thought of as "light-weight threads" without the resource penalties of full operating system threads. This bears some resemblance to Java's Fork/Join. How does Kotlin provide this alternate means of execution? A deeper understanding of what's going on under the surface is warranted.

6.3.1 How coroutines work

Let's start from the following modified example of what we saw in chapter 9:

```
package com.wgjd

import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

fun main() {
  GlobalScope.launch {
    delay(1000)
    sus()
  }

  Thread.sleep(2000)
}

suspend fun sus() {
  println("Totally sus...")
}
```

Here we use the `GlobalScope.launch` to start a new coroutine. *Scopes* are used to express how a coroutine should be run, and we'll look at them more carefully in the next section.

The coroutine we create here will wait for a second using the `delay` function, and then call our own function, `sus`. Lastly, we `Thread.sleep` for 2 seconds to make sure our coroutine has time to complete before our whole program exits.

Part of how coroutines differ from threads is that their execution may be paused at specific points. How does Kotlin know where it can pause? That comes from the `suspend` keyword we put on our `sus` function (and also on the library-provided `delay` function). The `suspend` function marks off the blocks of code that Kotlin treats as the units for execution.

With `suspend` functions marking the chunks of code in our coroutine between which we can pause, Kotlin can create a *state machine* to manage our coroutine's execution. Progress through that state machine is tracked by our generated code, with our `suspend`-defined blocks providing the steps to that machine.

Let's translate our coroutine into a state machine like this:

```
GlobalScope.launch {
  delay(1000)
  sus()
}
```

The steps in our coroutine follow and are illustrated in figure 16.4:

- Create our new coroutine instance by calling `launch`.
- Execute through `delay`.
- Control returns to Kotlin, which will wait the requested 1 second.
- Resume after `delay` and execute through `sus`.
- Hand control back to Kotlin, without requiring a pause this time.
- Resume after `sus`, and finish the coroutine.



Figure 16.4 Coroutine state machine

This breakdown into steps clarifies why coroutines are sometimes described as "cooperative multitasking." The code between our calls to `suspend` functions is executed synchronously, with the `suspend` points providing the only opportunity to pause and execute other coroutine steps. Imagine between `delay` and `sus` we entered an infinite loop. That loop would block whatever thread it's executing on forever.

This state machine isn't just an idea—Kotlin directly generates code that does this, and we can examine it. Let's look at the output from our previous function to see how it gets translated. (Note, some minor details are excluded for length and clarity.)

The compilation results from our basic app has more class files than we might expect. For example, here's the resulting output under Gradle's build directory:

```
build
└── classes
    └── kotlin
        └── main
            └── com
                └── wgjd
                    ├── MainKt$main$1.class
                    └── MainKt.class
```

We encountered the `MainKt.class` in chapter 9. Kotlin transparently creates a holding class for top-level functions, because the JVM doesn't natively support free-floating functions and must place all method code in some class.

Alongside that, though, we have a new class: `MainKt$main$1.class`. Disassembling that class file reveals that Kotlin has done similar work creating "secret" classes for our coroutine, just like it did for our top-level functions. This generated class represents a single execution of our coroutine. As we can see next, this generated class is a mixture of our code and the plumbing to run what we've written as a coroutine:

```
final class com.wgjd.MainKt$main$1
  extends kotlin.coroutines.jvm.internal.SuspendLambda    (1)
  implements kotlin.jvm.functions.Function2<              (2)
    kotlinx.coroutines.CoroutineScope,
    kotlin.coroutines.Continuation<? super kotlin.Unit>,
    java.lang.Object> {
```

(1) Our generated class gets functionality from Kotlin's SuspendLambda.

❷ Our generated class implements a specific interface that calling code will use to invoke our coroutine.

If we check out the code generated from our `main` function in `MainKt.class`, we see Kotlin created an instance of our coroutine and then invoked it:

```
Compiled from "Main.kt"
public final class com.wgjd.MainKt {

  public static final void main();
    Code:
       0: getstatic     #41  // Field                                  ❶
                         // kotlinx/coroutines/GlobalScope.INSTANCE:
                         // Lkotlinx/coroutines/GlobalScope;
       3: checkcast     #43  // class kotlinx/coroutines/CoroutineScope
       6: aconst_null
       7: aconst_null
       8: new           #45  // class com/wgjd/
                         // MainKt$main$1                              ❷
      11: dup
      12: aconst_null
      13: invokespecial #49  // Method com/wgjd/MainKt$main$1."<init>":
                         // (Lkotlin/coroutines/Continuation;)V
      16: checkcast     #51  // class kotlin/jvm/functions/Function2
      19: iconst_3
      20: aconst_null
      21: invokestatic  #57  // Method kotlinx/coroutines/
                         // BuildersKt.launch$                         ❸
                         // default:(Lkotlinx/coroutines/CoroutineScope;
                         // Lkotlin/coroutines/CoroutineContext;
                         // Lkotlinx/coroutines/CoroutineStart;
                         // Lkotlin/jvm/functions/Function2;
                         // ILjava/lang/Object;)Lkotlinx/coroutines/Job;
      24: pop
      25: ldc2_w        #58  // long 2000l
      28: invokestatic  #65  // Method java/lang/Thread.sleep:(J)V
      31: return
```

❶ Retrieves the GlobalScope instance to use starting our coroutine

❷ Creates and initializes a new instance of our generated coroutine class

❸ Calls the launch method, providing it the scope and our coroutine instance. In particular, our coroutine instance is passed as the Function2 parameter.

The code in the `launch` method will then start calling methods on our generated coroutine instance, running our state machine. Let's look at the bytecode implementing that state machine. First up, the coroutine instance has two separate fields. These track the coroutine scope and the current location in our state machine:

```
final class com.wgjd.MainKt$main$1
   extends kotlin.coroutines.jvm.internal.SuspendLambda
   implements kotlin.jvm.functions.Function2<
      kotlinx.coroutines.CoroutineScope,
      kotlin.coroutines.Continuation<? super kotlin.Unit>,
      java.lang.Object> {

  java.lang.Object L$0;    ❶

  int label;               ❷
```

❶ CoroutineScope for our current execution

❷ The int value representing our state machine's current step

When our state machine runs, the heart of it is a method that decides our next step based on these fields. Kotlin generates a method called `invokeSuspend` for just that purpose. `invokeSuspend` ends up as a mixture of our code and the state machine tracking our progress. During the lifetime of our coroutine, Kotlin will repeatedly call `invokeSuspend` whenever the coroutine is ready for its next execution step.

The next code snippet shows us the beginning of `invokeSuspend`, along with our first step from the state machine (from the coroutine's start until we call `delay`):

```
public final java.lang.Object invokeSuspend(java.lang.Object);
   Code:
     0: invokestatic  #36  // Method kotlin/coroutines/intrinsics/
                           // IntrinsicsKt.getCOROUTINE_SUSPENDED:
                           // ()Ljava/lang/Object;
     3: astore_3
     4: aload_0
     5: getfield      #40  // Field label:I                    ❶
     8: tableswitch   {    // 0 to 2
                 0: 36
                 1: 69
                 2: 104
           default: 122
```

```
    }
  36: aload_1                                    ❷
  37: invokestatic  #46  // Method kotlin/ResultKt.throwOnFailure:
                    // (Ljava/lang/Object;)V
  40: aload_0
  41: getfield      #48  // Field p$:Lkotlinx/coroutines/CoroutineScope;
  44: astore_2
  45: ldc2_w        #49  // long 1000l
  48: aload_0
  49: aload_0
  50: aload_2
  51: putfield      #52  // Field L$0:Ljava/lang/Object;
  54: aload_0
  55: iconst_1
  56: putfield      #40  // Field label:I
  59: invokestatic  #58  // Method kotlinx/coroutines/DelayKt.delay:
                    // (JLkotlin/coroutines/Continuation;)
                    // Ljava/lang/Object;
  62: dup
  63: aload_3
  64: if_acmpne     82
  67: aload_3
  68: areturn
  69: aload_0                                    ❸
  70: getfield      #52  // Field L$0:Ljava/lang/Object;
  73: checkcast     #60  // class kotlinx/coroutines/CoroutineScope

  // Further steps excluded for length.
  // See resources for full listing
```

❶ Determines our next step in the state machine

❷ The beginning of first step (up to the delay call)

❸ The beginning of second step (up to the sus call), which will happen when invokeSuspend is called again

After gathering information on the current coroutine, byte 5 loads up what step we're executing next from the `label` field. Then at byte 8, it uses an opcode we haven't seen before, `tableswitch`. This opcode looks at the value on the stack and jumps based on the values defined. Because this is our first time through `invokeSuspend`, our `label` has a value of 0, and we proceed to byte 36. We execute linearly from there. At bytes 55 and 56, we update our state `label` to 1 and advance to the next step. We call `delay` at byte 59 and then `areturn` from `invokeSuspend` at byte 68.

At this point Kotlin's code is given control, and it decides when to run the coroutine's next step. When it determines the time is right, it will call `invokeSuspend` on the same coroutine instance. Our state `label` will be set to 1, and we jump ahead to execute the code for our second step, after `delay` but before our `sus` call.

Although it's not truly necessary to get down to the bytecode level for day-to-day use of coroutines, understanding the mechanism is valuable. The well-grounded developer also shouldn't be satisfied when a feature seems too magical. At the end of the day, it's all just code executing one instruction at a time, and we have the tools to understand it.

This examination also may answer some questions from section 15.3.7 where we saw the `yield` function used in defining a Kotlin sequence. We hand-waved a little at that point about how our lambda to define the sequence could "pause" its execution. This is, in fact, using the same mechanism of a generated state machine based on `suspend` functions. Each successive calls to `invokeSuspend` gets the next item in the sequence.

6.3.2 Coroutine scoping and dispatching

Although coroutines provide a different abstraction to the standard operating system threading model, under the surface our code is still executing in a thread somewhere. How that work gets distributed and coordinated by Kotlin's coroutines is managed by coroutine scopes and *dispatchers*.

Let's revise our example to peek at where each step of our coroutine is actually executing:

```
package com.wgjd

import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

fun main() {
  GlobalScope.launch {
    println("On thread ${Thread.currentThread().name}")
    delay(500)

    println("On thread ${Thread.currentThread().name}")
    delay(500)

    println("On thread ${Thread.currentThread().name}")
  }
```

```
  Thread.sleep(2000)
}
```

The results aren't deterministic but will look something like this:

```
On thread DefaultDispatcher-worker-1
On thread DefaultDispatcher-worker-2
On thread DefaultDispatcher-worker-1
```

Our thread name provides two interesting pieces of information: a name for a dispatcher (`DefaultDispatcher`) and a number indicating which thread from an available pool we're using. When we asked for a particular scope—in our case, the `GlobalScope` , which lasts the entire application lifetime—part of what we're choosing is the dispatcher, which determines how our work is actually scheduled.

Let's say that we want greater control over how that work dispatching happens. Rather than using `GlobalScope` , we can create our own, separate `CoroutineScope` instance, as shown next. Typically, a scope is associated with a different object in our system with its own specific lifetime. Our custom scopes require a context at construction, and the standard functions factory methods are largely identified by how they configured dispatching:

```
val context: CoroutineContext = newFixedThreadPoolContext(3, "New-Pool")

CoroutineScope(context).launch {
  println("On thread ${Thread.currentThread().name}")
  delay(500)

  println("On thread ${Thread.currentThread().name}")
  delay(500)

  println("On thread ${Thread.currentThread().name}")
}
```

The not terribly surprising output shows we're executing on an entirely separate set of threads than we saw before:

```
On thread New-Pool-1
On thread New-Pool-2
```

```
On thread New-Pool-1
```

NOTE It's worth paying attention because the `newFixedThreadPoolContext` and other related functions are marked as obsolete, but their replacements are not available as of this writing. Check out the Kotlin coroutine documentation at **https://kotlin.github.io/kotlinx.coroutines** for the most up-to-date practices.

Context classes wrap up more than just a dispatcher for coroutines. Examples of additional information we can provide are a name for our coroutine (which improves the debugging experience in IDEs, particularly when many coroutines share dispatchers) and a generalized error handler. Elements are added to an existing context object using the `plus` function, as shown here:

```
val context: CoroutineContext = newFixedThreadPoolContext(3, "New-Pool")
  .plus(CoroutineExceptionHandler { _, thrown ->
     println(thrown.message + "!!!!") })
  .plus(CoroutineName("Our Coroutine"))

CoroutineScope(context).launch {
  throw RuntimeException("Failed")
}
```

Alongside automatic cancellation of coroutines when the exception is raised, our `CoroutineExceptionHandler` will be run and print "Failed!!!!". You may wish to consider a more thorough error-handling strategy for your production applications, but coroutines provide the necessary hooks.

It's not unusual to want to execute various steps in parallel but then require the results from those stages before continuing. Within a coroutine this is supported by the `async` function, as shown next, which returns a `Deferred<T>` — effectively a coroutine `Job` that also allows for waiting and retrieving the value:

```
GlobalScope.launch {
   val result: Deferred<Int> = async {
     10;
   }

   println("Got ${result.await()}")    ❶
 }
```

❶ Unsurprisingly, prints Got 10

Back in section 9.5, we saw that by default, coroutines will cancel the entire coroutine hierarchy on errors as follows:

```
val failed = GlobalScope.launch {
 launch { throw RuntimeException("Failing...") }
}

Thread.sleep(2000)                    ❶

println("Cancelled ${failed.isCancelled}")    ❷
```

❶ Gives time to finish executing

❷ Prints Cancelled true

When you want it, this behavior is extremely powerful. However, it may not always be desirable. If some child coroutines can safely fail, we can wrap them with the `supervisorScope` as follows. This behaves like a typical coroutine wrapper, but doesn't percolate up cancellations:

```
val supervised = GlobalScope.launch {
 supervisorScope {
  launch { throw RuntimeException("Failing...") }
 }
}

Thread.sleep(2000)                    ❶

println("Cancelled ${supervised.isCancelled}")  ❷
```

❶ Again, needs time to finish

❷ Prints Cancelled false, indicating our supervisor allowed the child coroutine to fail without cancelling the parent

Coroutines provide us with many options for how we handle concurrent execution. They are far from the only alternate way of thinking about the world, though. Let's see what Clojure brings to the table.

16.4 Concurrent Clojure

Java's model of state is based fundamentally around the idea of mutable objects. As we saw in chapter 5, this leads directly to problems with safety in concurrent code. We need to introduce quite complicated locking strategies to prevent other threads from seeing intermediate (aka *inconsistent*) object states while a given thread is working on mutating an object's state. These strategies are hard to come up with, hard to debug, and harder to test.

Clojure takes a different view, and its abstractions for concurrency aren't as low-level as Java's in some respects. For example, the use of thread pools that are managed by the Clojure runtime (and over which the developer has little or no control) may seem strange. But the power gained stems from allowing the platform (in this case, the Clojure runtime) to perform the bookkeeping for you, allowing you to free up your mind for much more important tasks, such as overall design.

Overall, the philosophy guiding Clojure is to isolate threads from each other by default, which goes a long way to making the language concurrently typesafe by default. By assuming a baseline of "nothing needs to be shared" and having immutable values, Clojure sidesteps a lot of Java's issues and instead can focus on ways to share state safely for concurrent programming.

NOTE To help promote safety, Clojure's runtime provides mechanisms for coordinating between threads, and it's very strongly recommended that you use these mechanisms rather than trying to use Java idioms or making your own concurrency constructs.

Let's look at the first of these building blocks: persistent data structures.

6.4.1 Persistent data structures

A *persistent data structure* is one that preserves previous versions when it is modified. As a result, they are thread safe, because operations on them do not mutate the structure as seen by existing readers and instead always yield a new updated object.

All of the Clojure collections are persistent and allow for the efficient creation of modified copies, by using *structural sharing*. The collections are inherently thread safe and designed to be efficient.

It's important to note that Clojure's persistent collections do not permit in-place mutation or deletion of elements. They will throw `UnsupportedOperationException` if your program calls those methods of the Java Collections interfaces (such as `List` or `Map`).

Instead, the expectation is that the persistent collections will be built up using operations like `(cons)` and `(conj)`, in the Lisp tradition.

All the collections support the following basic methods:

- `count` gets the size of the collection.
- `cons` and `conj` add to the collection.
- `seq` gets a sequence that can walk the collection.

All of the sequence functions can therefore be used with any collection, via the support for `seq`. Let's look at an example—Clojure's `PersistentVector`—starting with some Java code to show how the vector is build up by repeatedly adding new elements with `(cons)`:

```
var aList = new ArrayList<PersistentVector>();
var vector = PersistentVector.EMPTY;
for (int i=0; i < 32; i = i + 1) {
   vector = vector.cons(i);
   aList.add(vector);
}
System.out.println(aList);
```

This will output something like

```
[[0], [0 1], [0 1 2], [0 1 2 3],

...

[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
 28 29 30 31]]
```

which shows that it is possible to keep each earlier version of the persistent vector around, should you need to. This would also mean that it would be possible to pass the vector between threads and have each thread modify it without affecting the others.

Let's take a quick look at how this data structure is implemented. Recall that array-backed data structures (such as vectors) in other languages are usually implemented in terms of a single continuous chunk of memory. This implementation makes indexed operations, such as lookup, fast, but for operations such as making a mutated copy of the vector (while retaining the original), we must copy the entire backing array.

Clojure's `PersistentVector` is very different. Instead, Clojure stores elements for the vector in chunks of 32 elements. The general idea is that if an element is added, only the current *tail* of 32 elements needs to be copied. If a vector has more than 32 elements added to it, a structure called a *node* is created that contains an array of 32 elements, which are themselves references to nodes that contain full arrays of 32 elements.

NOTE `PersistentVector` is one of Clojure's core abstractions, and it is so widely used that is actually implemented in Java (not Clojure), because it is needed to bootstrap the basic Clojure language runtime.

The class definition looks like this (slightly simplified for clarity):

```
public class PersistentVector extends APersistentVector
                        implements IObj, ... {

   // ...

   public final PersistentVector.Node root;
   public final Object[] tail;
   private final int cnt;

   // ...

   public static final PersistentVector EMPTY;
   public static final PersistentVector.Node EMPTY_NODE;

  // ...
}
```

with the inner class `PersistentVector.Node` defined as follows:

```
public static class Node implements Serializable {
      public final transient AtomicReference<Thread> edit;
      public final Object[] array;

      public Node(AtomicReference<Thread> edit, Object[] array) {
         this.edit = edit;
         this.array = array;
      }

      Node(AtomicReference<Thread> edit) {
         this.edit = edit;
         this.array = new Object[32];
```

```
    }
  }
```

Note that the `array` field is just an `Object[]`. This is an aspect of Clojure's dynamically typed nature—there are no generics here.

In addition, Clojure makes use of public final fields frequently in its core and does not always define accessor methods. We can therefore peek inside the data structure and see how the nodes work. For example:

```
var vector = PersistentVector.EMPTY;
for (int i = 0; i < 32; i = i + 1) {
   vector = vector.cons(i);
}
System.out.println(Arrays.toString(vector.tail));
System.out.println(Arrays.toString(vector.root.array));
System.out.println("----------------");
for (int i=32; i < 64; i = i + 1) {
   vector = vector.cons(i);
}
System.out.println(Arrays.toString(vector.tail));
System.out.println(Arrays.toString(vector.root.array));
var earlier = (PersistentVector.Node)(vector.root.array[0]);
System.out.println("Earlier: "+ Arrays.toString(earlier.array));
```

produces output like this:

```
[0, 1, 2, 3, ... 31]
[null, null, null, null, ... null]
---------------
[32, 33, 34, 35,  ...  63]
[clojure.lang.PersistentVector$Node@783e6358, null, null, null, ... null]
Full Tail: [0, 1, 2, 3, ... 31]
```

After 64 elements have been added, the `tail` is `[32, ... 63]`, and `root.array` contains a single `PersistentVector.Node` that contains the elements `[0, ... 31]` as its array field. So, in pictorial form, for 0–32 elements, the vector looks like figure 16.5.

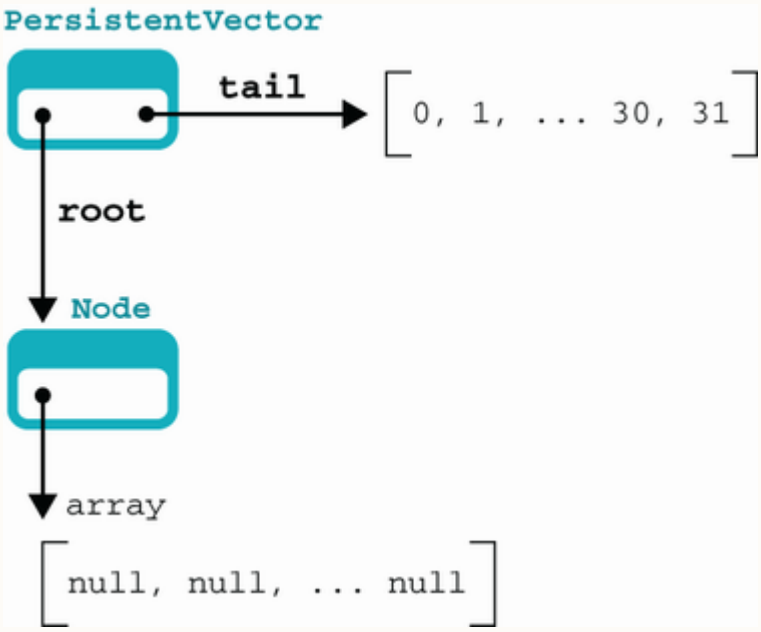


Figure 16.5 A persistent vector with 32 elements

For a number of elements more than 32 and less than 64, the structure will look like figure 16.6.

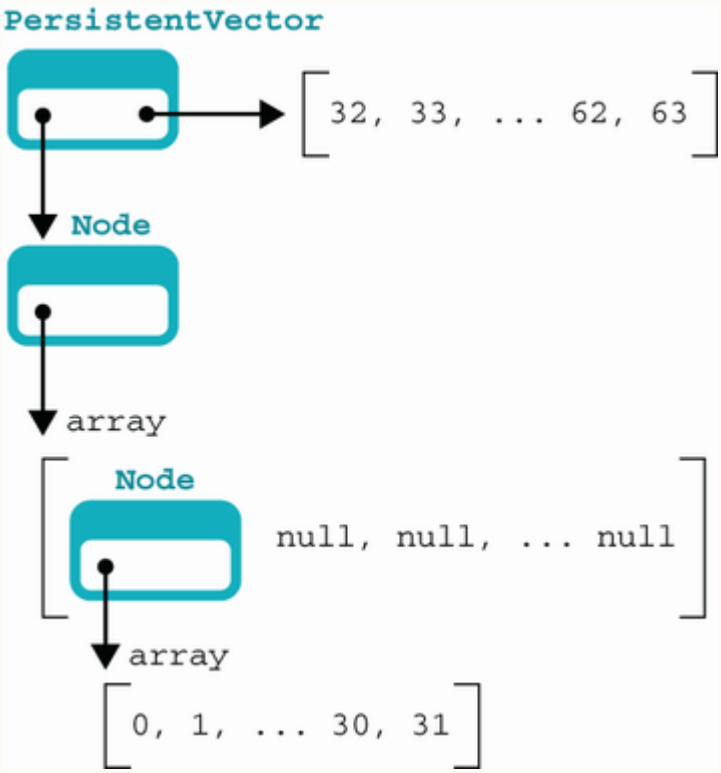


Figure 16.6 A persistent vector with 64 elements

And for 64 to 96 elements, it will look like it does in figure 16.7 and so on.

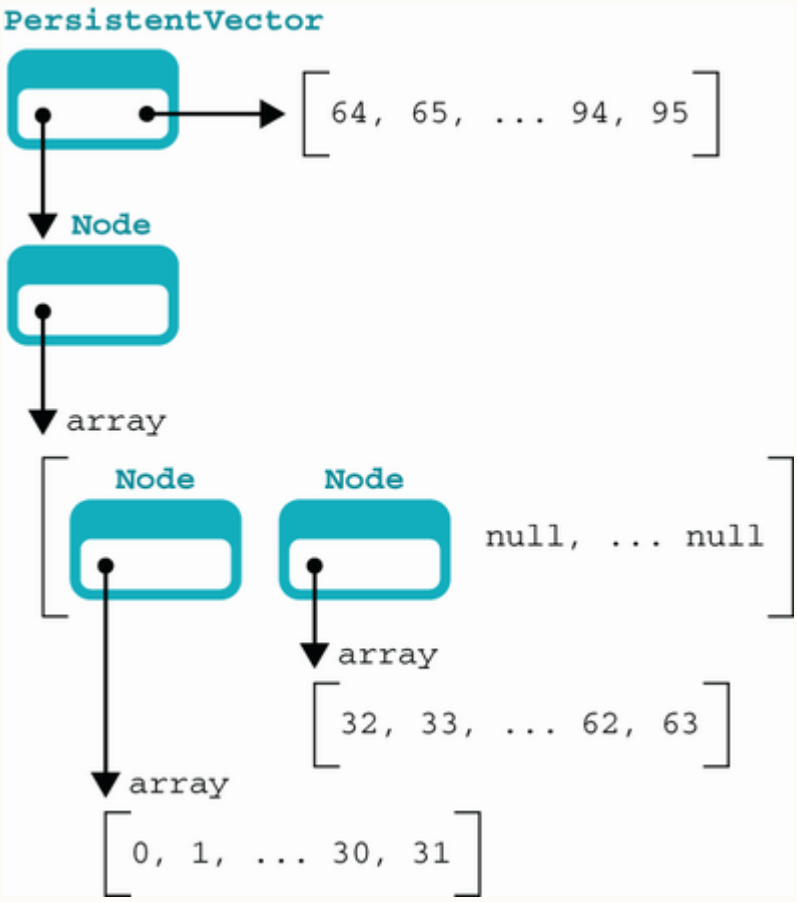


Figure 16.7 A persistent vector with 96 elements

You might, legitimately, ask what happens when all of the node's array slots are full. This occurs when the vector contains 32 + (32 * 32)—or 1,056—elements. The reader might be expecting this number to be 1,024, but we also have the *tail*, which contains 32 elements—effectively an "off-by-one effect." A full `PersistentVector` with one level of `Node` can be seen in figure 16.8.

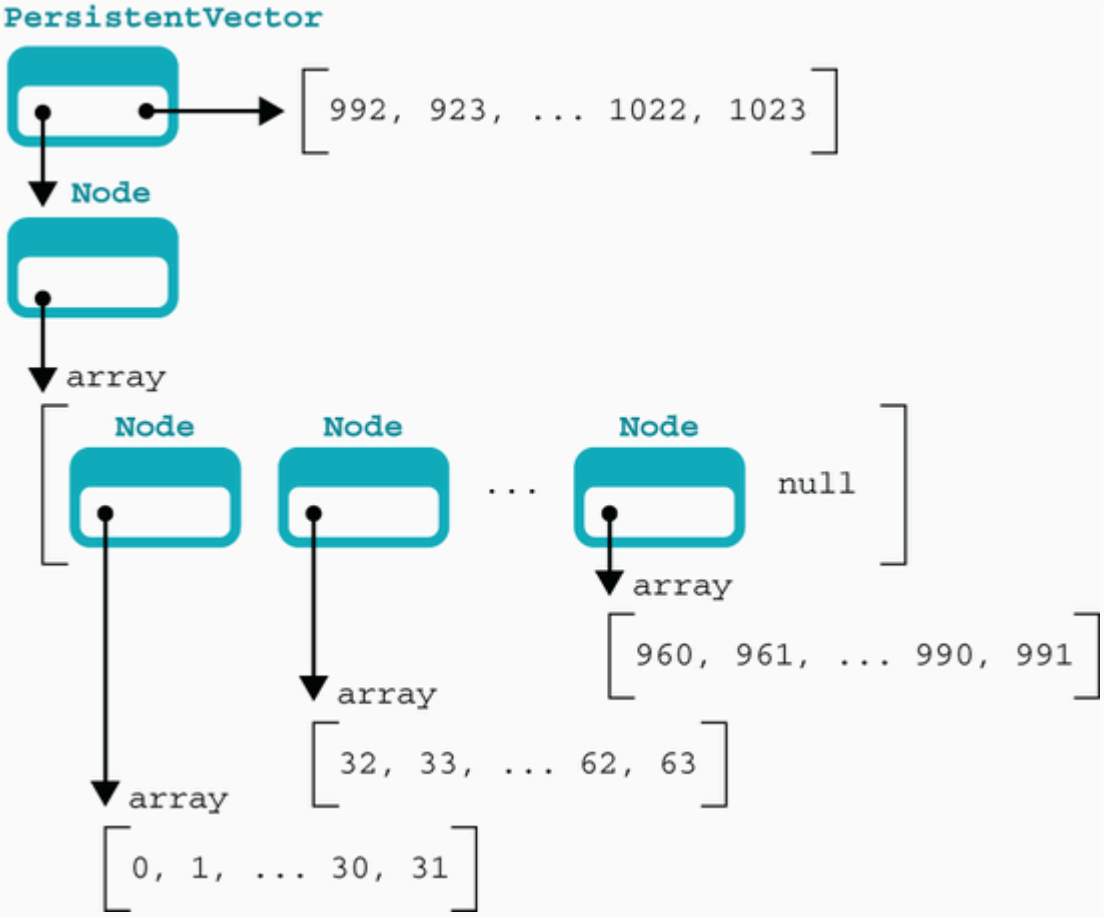


Figure 16.8 A persistent vector with 1,056 elements

If we continue to add elements, the tree structure grows another level, as shown in figure 16.9.

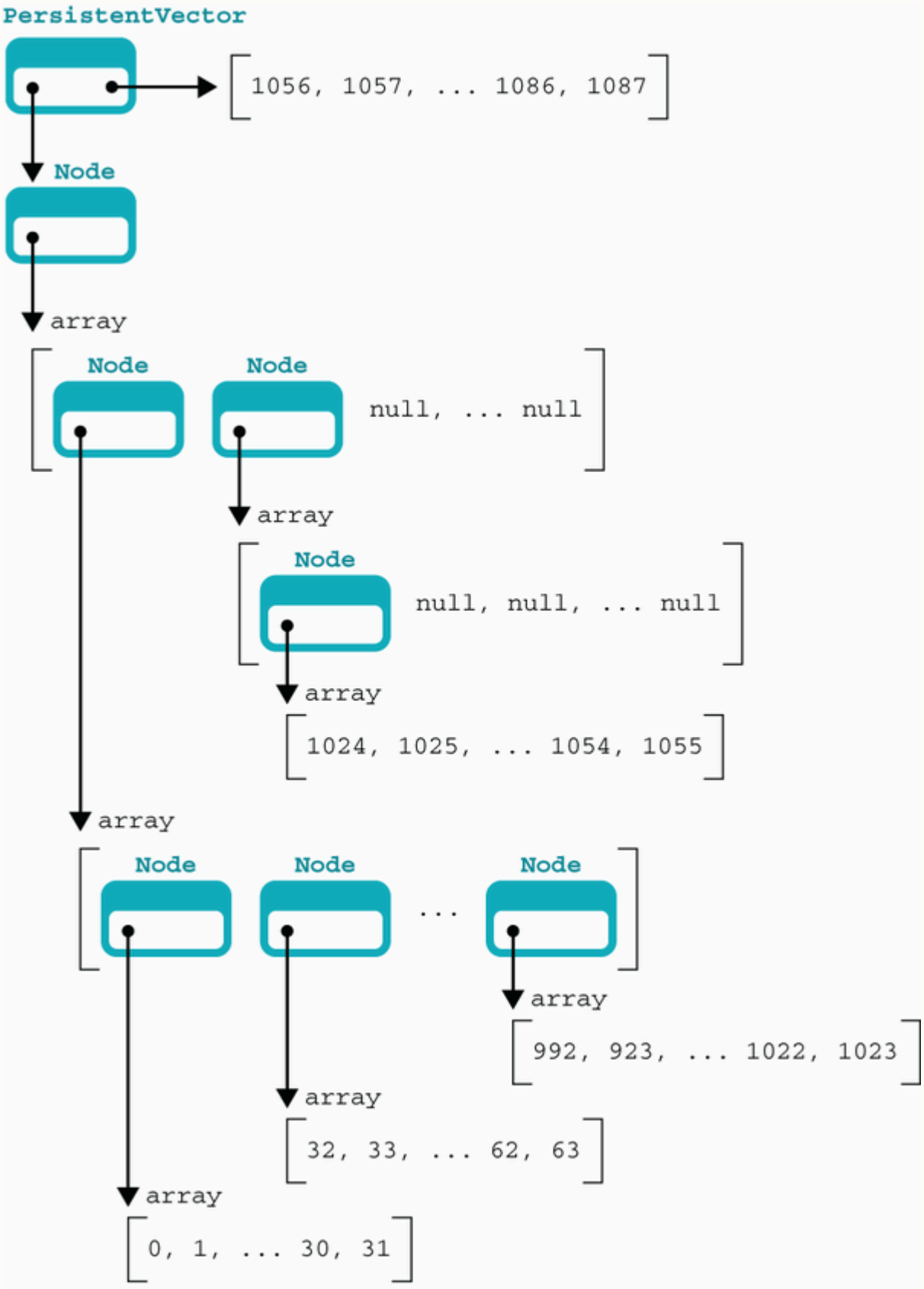


Figure 16.9 A persistent vector with many elements

To see this in code, we can run some code like this:

```
var vector = PersistentVector.EMPTY;
for (int i = 0; i < 1088; i = i + 1) {
   vector = vector.cons(i);
}
System.out.println(Arrays.toString(vector.tail));
```

```
System.out.println(Arrays.toString(vector.root.array));
System.out.println();

var v0 = (PersistentVector.Node) (vector.root.array[0]);
var v1 = (PersistentVector.Node) (vector.root.array[1]);
System.out.println("r.a[0] : " + Arrays.toString(v0.array));
System.out.println("r.a[1] : " + Arrays.toString(v1.array));
System.out.println();

var v0A0 = (PersistentVector.Node)(((PersistentVector.Node)v0).array[0]);
var v0A31 = (PersistentVector.Node)(((PersistentVector.Node)v0).array[31]);
var v1A0 = (PersistentVector.Node)(((PersistentVector.Node)v1).array[0]);
System.out.println("r.a[0].a[0] : " + Arrays.toString(v0A0.array));
System.out.println("r.a[0].a[31] : " + Arrays.toString(v0A31.array));
System.out.println("r.a[1].array[0] : " + Arrays.toString(v1A0.array));
```

This produces output like this:

```
[1056, 1057, 1058, 1059, ... 1087]
[clojure.lang.PersistentVector$Node@2344fc66,
clojure.lang.PersistentVector$Node@458ad742, null, null, ... null]

r.a[0] : [clojure.lang.PersistentVector$Node@735f7ae5,
        clojure.lang.PersistentVector$Node@180bc464, ... ,
        clojure.lang.PersistentVector$Node@617c74e5]
r.a[1] : [clojure.lang.PersistentVector$Node@6ea12c19, null, ... , null]

r.a[0].a[0] : [0, 1, ... , 31]
r.a[0].a[31] : [992, 993, ... , 1023]
r.a[1].a[0] : [1024, 1025, ... , 1055]
```

Notice how in this example, the array field of `v0` and `v1` no longer contains `Integer` elements but instead now contains `Node` objects. This is why the `array` field is typed as `Object[]`, to allow for this sort of dynamic typing.

NOTE If we continue adding elements, we will build a multilevel structure, and this allows `PersistentVector` to handle vectors of any size (albeit at an increasing cost of indirection for larger vectors).

However, data structures aren't the only things that Clojure needs to be a successful concurrent language. For example, a notion of concurrency models and execution is ab-

solutely essential. Fortunately, Clojure has you covered!

In fact, Clojure uses several methods to provide different sorts of concurrency models: futures and pcalls, refs, and agents. Let's look at each in turn, starting with one of the simplest.

6.4.2 Futures and pcalls

We should start by making it clear that you can always start new threads by exploiting Clojure's tight binding to Java. Anything you can do in Java, you can also do in Clojure, and you can write concurrent Java code very easily in Clojure.

However, some of Java's abstractions have a cleaned-up form within Clojure. For example, Clojure provides a very clean approach to the `Future` concept that we encountered in Java in chapter 6. The following listing shows a simple example.

Listing 16.2 Futures in Clojure

```
user=> (def simple-future
 (future (do
     (println "Line 0")
     (Thread/sleep 10000)
     (println "Line 1")
     (Thread/sleep 10000)
     (println "Line 2"))))

#'user/simple-future
Line 0                          ❶
user=> (future-done? simple-future)
user=> false
Line 1
user=> @simple-future           ❷
Line 2
nil
user=>
```

❶ Starts executing at once

❷ Blocks when dereferencing

In this listing, you set up a Future with `(future)`. As soon as this is created, it begins to run on a background thread, which is why you see the printout of line 0 (and later line 1)

on the Clojure REPL—the code has started to run on another thread.

You can then test to see whether the code has completed using `(future-done?)`, which is a nonblocking call (like `isDone()` in Java). The attempt to dereference the future, however, causes the calling thread to block until the function has completed.

This is effectively a thin Clojure wrapper over a Java Future, with some slightly cleaner syntax. Clojure also provides helper forms that can be very useful to the concurrent programmer. One simple function is `(pcalls)`, which takes in a variable number of zero-argument functions and executes them in parallel.

NOTE `(pcalls)` is somewhat similar to Java's `ExecutorService.invokeAll()` helper method.

The calls are executed on a runtime-managed threadpool and will return a lazy seq of the results. Trying to access any elements of the seq that haven't yet completed will cause the accessing thread to block.

Listing 16.3 sets up a one-argument function called `(wait-with-for)`. This uses a loop form similar to the one introduced in section 10.2.5. From this, you create a number of zero-argument functions—`(wait-1)`, `(wait-2)`, and so on—which you can feed to `(pcalls)`.

Listing 16.3 Parallel calls in Clojure

```
user=> (defn wait-with-for [limit]
  (let [counter 1]
    (loop [ctr counter]
      (Thread/sleep 500)
      (println (str "Ctr=" ctr))
    (if (< ctr limit)
      (recur (inc ctr))
    ctr))))
#'user/wait-with-for

user=> (defn wait-1 [] (wait-with-for 1))
#'user/wait-1

user=> (defn wait-2 [] (wait-with-for 2))
#'user/wait-2

user=> (defn wait-3 [] (wait-with-for 3))
#'user/wait-3
```

```
user=> (def wait-seq (pcalls wait-1 wait-2 wait-3))
#'user/wait-seq
Ctr=1
Ctr=1
Ctr=1
Ctr=2
Ctr=2
Ctr=3

user=> (first wait-seq)
1

user=> (first (next wait-seq))
2
```

With a thread sleep value of only 500 ms, the wait functions complete very quickly. By playing with the timeout (such as by extending it to 10 seconds), it's easy to verify that the lazy sequence called `wait-seq` that is returned by `(pcalls)` has the described blocking behavior.

This access to simple multithreaded constructs is fine for the case where you don't need to share state, but in many applications, different processing threads need to communicate in flight. Clojure has a couple of models for handling this, so let's look at one of these next: the shared state enabled by the `(ref)` form that is handled in a transaction.

6.4.3 Software transactional memory

The first, and most obvious, way to share state is not to. In fact, the Clojure construct that we've been using up until now, the `var`, isn't really able to be shared. If two different threads inherit the same name for a `var` and rebind it in-thread, then those rebindings are visible only within those individual threads and can never be shared by other threads.

This is by design, and Clojure provides an alternative way of sharing state between threads: the *ref*. This concept relies upon a model provided by the runtime for state changes that need to be seen by multiple threads. The model effectively introduces an additional level of indirection between a symbol and a value—that is, a symbol is bound to a reference to a value, rather than directly to a value.

The system is essentially transactional, and changes to the underlying value are coordinated by the Clojure runtime. This is illustrated in figure 16.10.

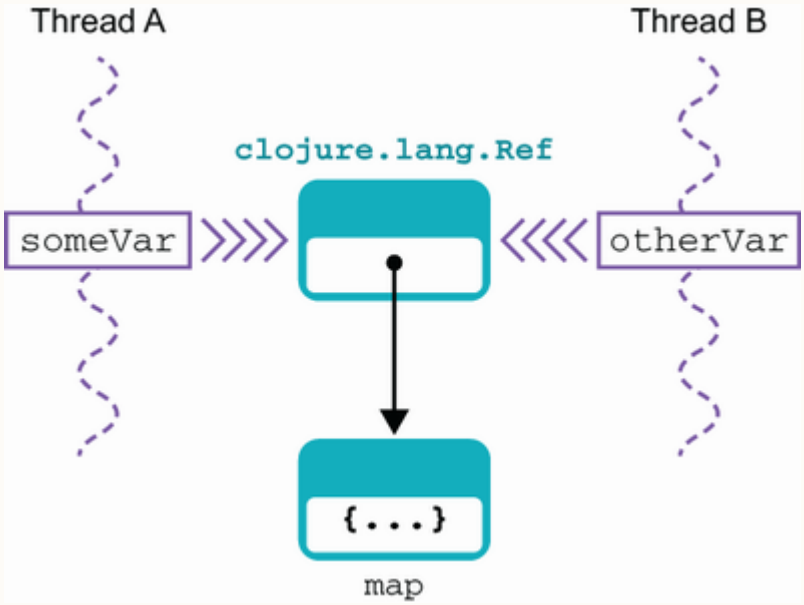


Figure 16.10 Software transactional memory

This indirection means that before a ref can be altered or updated, it has to be placed inside a transaction. When the transaction is completed, either all or none of the updates will take effect, and this has an obvious analogy to a transaction in a database.

This can seem a bit abstract, so let's return to an earlier example, the `Account` classes we discussed in chapters 5 and 6. Recall that, to avoid concurrency problems like lost update, in Java you're required to protect every sensitive bit of data with locks, as shown here:

```
// ...

private final Lock lock = new ReentrantLock();
private int balance;

public boolean withdraw(final int amount) {
   // Elided code - check to see amount > 0, throw if not

   lock.lock();
   try {
      if (balance >= amount) {
         balance = balance - amount;
         return true;
      }
   } finally {
      lock.unlock();
   }
   return false;
}
```

```
    // ...
```

Let's see how you could try to write something similar to this in Clojure. However, it's here that we run into something of a conceptual problem.

In Java, the default is to use mutable state, and this is the case in the previous code. The `withdraw()` method takes in a single parameter, `amount`, and one of the following three things happens:

- `amount` *is less than or equal to zero* —An `IllegalArgumentException` is thrown, because this is not a valid withdrawal.
- *The withdrawal succeeds*—The balance is updated, and `true` is returned.
- *The withdrawal fails (not enough money available)*—The balance is *not* updated, and `false` is returned.

Leaving aside the invalid case, two separate aspects are happening here: updating mutable state and signaling via the return code whether or not the operation succeeded.

In functional programming, rather than updating mutable state, we would usually return a new value that contains the updated state. However, if the withdrawal fails, how will the user of the code know whether the balance has been updated? We could imagine changing the return code to be a pair of return codes and possibly an updated value, but this is somewhat clumsy.

Instead, let's start with a slightly different single-threaded version shown in the next listing. The semantics here are that the `(debit)` form operates on a map (representing an account) and returns a new map if the withdrawal succeeds—and throws if it fails.

Listing 16.4 Simple account model in Clojure

```
(defn make-new-acc [account-name opening-balance]
  {:name account-name :bal opening-balance})

(defn debit [account amount]
  (let [balance (:bal account) my-name (:name account)]
    (if (<= amount 0)
      (throw (AssertionError. "Withdrawal amount cannot be < 0")))
    (if (> balance amount)
      (make-new-acc my-name (- balance amount))
      (throw (AssertionError. "Withdrawal amount cannot exceed balance"))
```

```
  )))

(debit (make-new-acc "Ben" 5000) 1000)
```

Notice how compact this code is compared to the Java version. Admittedly, this is still single threaded, but it's a lot less code than was needed for Java. Running the code will give you the expected result: you end up with a map with a balance of 4000.

Despite the relative simplicity, this is not entirely satisfactory—we are actually solving a different problem with different semantics. Let's see if we can resolve some of the problems by generalizing to a concurrent version.

To make this code concurrent, we need to introduce Clojure's refs. These are created with the `(ref)` form and are JVM objects of type `clojure.lang.Ref`. Usually they're set up with a Clojure map to hold the state.

We'll also need the `(dosync)` form, which sets up a transaction. Within this transaction, we'll also use the `(alter)` form, which can be used to modify a ref's contents. Let's see how to use refs for this multithreaded approach to our account construct, as shown in the following.

Listing 16.5 Multithreaded account handling

```
user=> (defn safe-debit [ref-account amount]
  (dosync
    (alter ref-account debit amount)
    ref-account))
#'user/safe-debit

user=> (def my-acc (make-new-acc "Ben" 5000))
#'user/my-acc

user=> (def r-my-acc (ref my-acc))
#'user/r-my-acc

user=> (safe-debit r-my-acc 1000)
#object[clojure.lang.Ref 0x6b1e7ad3 {:status :ready,
                                     :val {:name "Ben", :bal 4000}}]
```

As noted, the `(alter)` form acts on a ref by applying a function with arguments. The value acted upon is the local value visible to this thread during the transaction. This is

called the *in-transaction value*. The value returned is the new value of the ref after the alter function returns. This value isn't visible outside the altering thread until you exit the transaction block defined by `(dosync)`.

Other transactions may be proceeding at the same time as this one. If so, the Clojure STM system will keep track of that and will allow a transaction to commit only if it's consistent with other transactions that have committed since it started. If it's inconsistent, it will be rolled back and may be retried with an updated view of the world.

This retry behavior can cause problems if the transaction does anything that produces side effects (such as a log file or other output). It's up to you to keep the transactional parts as simple and as pure in the functional programming sense (meaning as side-effect-free) as possible.

For some multithreaded approaches, this optimistic transactional behavior can seem to be a rather heavyweight approach. Some concurrent applications need to communicate between threads only occasionally and in a rather asymmetric fashion. Fortunately, Clojure provides another concurrency mechanism that is much more fire-and-forget, and it's the topic of our next section.

6.4.4 Agents

Agents are another of Clojure's concurrency primitives. Instead of using a shared state, a Clojure agent is an asynchronous, message-oriented execution object. They are similar to the *actor* concept in other languages (such as Scala and Erlang).

An agent is an execution context that can receive messages (in the form of functions) sent to it from another thread (or the same one). New agents are declared with the `(agent)` function and messages can be sent to them using `(send)`.

"They must go by the carrier," she thought; "and how funny it'll seem, sending presents to one's own feet! And how odd the directions will look!"

—Lewis Carroll

The agents are not threads themselves but are instead executable objects that are "smaller" than a thread. They are scheduled on a threadpool that is managed by the Clojure runtime (the threadpool isn't usually accessible to the programmer directly).

NOTE Clojure agents are potentially long-lived, unlike task objects for Java threadpools, which typically have bounded lifetimes.

The runtime also ensures that the values of the agent that are seen from outside are isolated and atomic. This means that user code will only see the value of the agent in its before or after state.

NOTE We will meet another example of executable objects smaller than a thread in chapter 18.

The following listing shows a simple example of agents, similar to the example we used to discuss futures.

Listing 16.6 Agents in Clojure

```
user=> (defn wait-and-log [coll str-to-add]
  (do (Thread/sleep 10000)
    (let [my-coll (conj coll str-to-add)]
      (Thread/sleep 10000)
      (conj my-coll str-to-add))))
#'user/wait-and-log

user=> (def str-coll (agent []))
#'user/str-coll

user=> (send str-coll wait-and-log "foo")
#object[clojure.lang.Agent 0x38499e48 {:status :ready, :val []}]

user=> @str-coll
[]

// Wait to allow message to be handled

user=> @str-coll
["foo" "foo"]
```

The `(send)` call dispatches a `(wait-and-log)` call to the agent, and by using the REPL to dereference it, you can see that, as promised, you can never see an intermediate state of the agent—only the final state appears (where the `"foo"` string has been added twice).

It could seem odd that in the agent approach, we're sending a message to an agent that's scheduled on a thread in a Clojure-managed threadpool, when both threads already share an address space. But one of the themes in concurrency that you've now encountered several times is that additional abstraction can be a good thing if it enables a simpler and clearer usage.

Nowhere is this synergy clearer than in Clojure's delegation of many low-level aspects of threading and concurrency control to the runtime. This frees the programmer to focus on good multithreaded design and higher-level concerns. This is analogous to the way in which Java's garbage collection facilities allow you to free yourself from the details of memory management.

Summary

- Each language has extended the core concepts of execution in their own way.
- Java has introduced decomposable tasks and work-stealing through the Fork/Join library; Kotlin uses advanced compiler tricks to produce a version of coroutines; and Clojure builds in a form of the actor model using the agent concept.
- The treatment of state differs between our three languages and is key to concurrent programming
- In Java, mutability is the default, with some enhancements such as `CompletableFuture`.
- Kotlin places a greater emphasis on immutability but is still fundamentally drawing from the same worldview of shared, mutable state.
- Clojure centers immutability, via software transactional memory, but at the cost of a less-familiar programming model and a less-close integration with the Java Collections.

Previous chapter
15 Advanced functional programming

Next chapter
17 Modern internals